



INDUSTRIAL DIESEL GENERATOR MONITORING AND MAINTENANCE SYSTEM

EXERCISE NO – 10

HARRISH T M	– 24MER016
KAPIL P	– 24MER022
RAMPRASATH D	– 24MER047
SARAVANAKARTHIK S R	– 24MER055
SETHUPATHI M	– 24MER056

10. Industrial Diesel Generator Monitoring and Maintenance System

Project Overview:

The Industrial Diesel Generator Monitoring and Maintenance System is an ESP8266-based monitoring system designed to continuously observe important electrical and environmental parameters of a diesel generator.

The system uses:

- ACS712 to measure current.
- Voltage Sensor Module to measure voltage.
- DHT11 to monitor temperature and humidity.
- OLED to display real-time information.
- Buzzer to provide an alarm when abnormal conditions are detected.
- ESP8266 NodeMCU to process all sensor data.

The system checks the measured parameters against predefined limits and determines whether the generator is operating normally or requires attention.

Main Parameters

- Voltage
- Current
- Temperature
- Humidity
- Electrical load
- Generator operating status
- Maintenance condition

Problem Statement:

Diesel generators are important backup power sources in industries, hospitals, construction sites, offices, and other facilities.

During operation, a generator can experience:

- High temperature
- Excessive electrical current
- Abnormal voltage
- High ambient humidity

- Electrical overload
- Poor operating conditions
- Lack of regular maintenance

Traditional monitoring often depends on manual inspection. This can result in delayed detection of abnormal conditions.

For example, if the generator is drawing excessive current, an operator may not immediately notice the problem. Continuous overload can potentially damage electrical equipment and reduce generator reliability.

Proposed Solution:

The proposed solution is to develop an ESP8266-based Industrial Diesel Generator Monitoring and Maintenance System that continuously monitors the generator's important operating and environmental parameters.

The system uses the ACS712 Current Sensor, Voltage Sensor Module, and DHT11 Sensor to collect real-time information. The ESP8266 NodeMCU processes the sensor readings and compares them with predefined safe operating limits. The current operating condition is displayed on an OLED display, while a buzzer provides an audible warning whenever an abnormal condition is detected.

- **Voltage monitoring:**
The voltage sensor measures the generator's electrical voltage. The ESP8266 checks whether the measured value is within the configured operating range.
- **Current monitoring:**
The ACS712 measures the current drawn by the connected load. If the current exceeds the configured rated-current limit, the system identifies a possible overload condition.
- **Temperature monitoring:**
The DHT11 measures the surrounding temperature. If the temperature rises above the defined warning level, the system generates a temperature warning.
- **Humidity monitoring:**
The DHT11 also measures humidity, allowing the operator to observe environmental conditions around the generator.
- **Data processing:**
The ESP8266 receives the sensor readings and continuously evaluates them against predefined threshold values.

System Architecture:

The System Architecture describes how the different hardware components of the Industrial Diesel Generator Monitoring and Maintenance System are connected and how information flows from the generator to the monitoring and alerting units.

The proposed system is divided into five major sections:

- Generator/Input Section
- Sensor/Measurement Section
- Processing and Control Section
- Display and Alert Section
- Maintenance Decision Section

he ESP8266 NodeMCU is the central processing unit of the system.

It receives sensor data and performs several operations.

Main functions of ESP8266

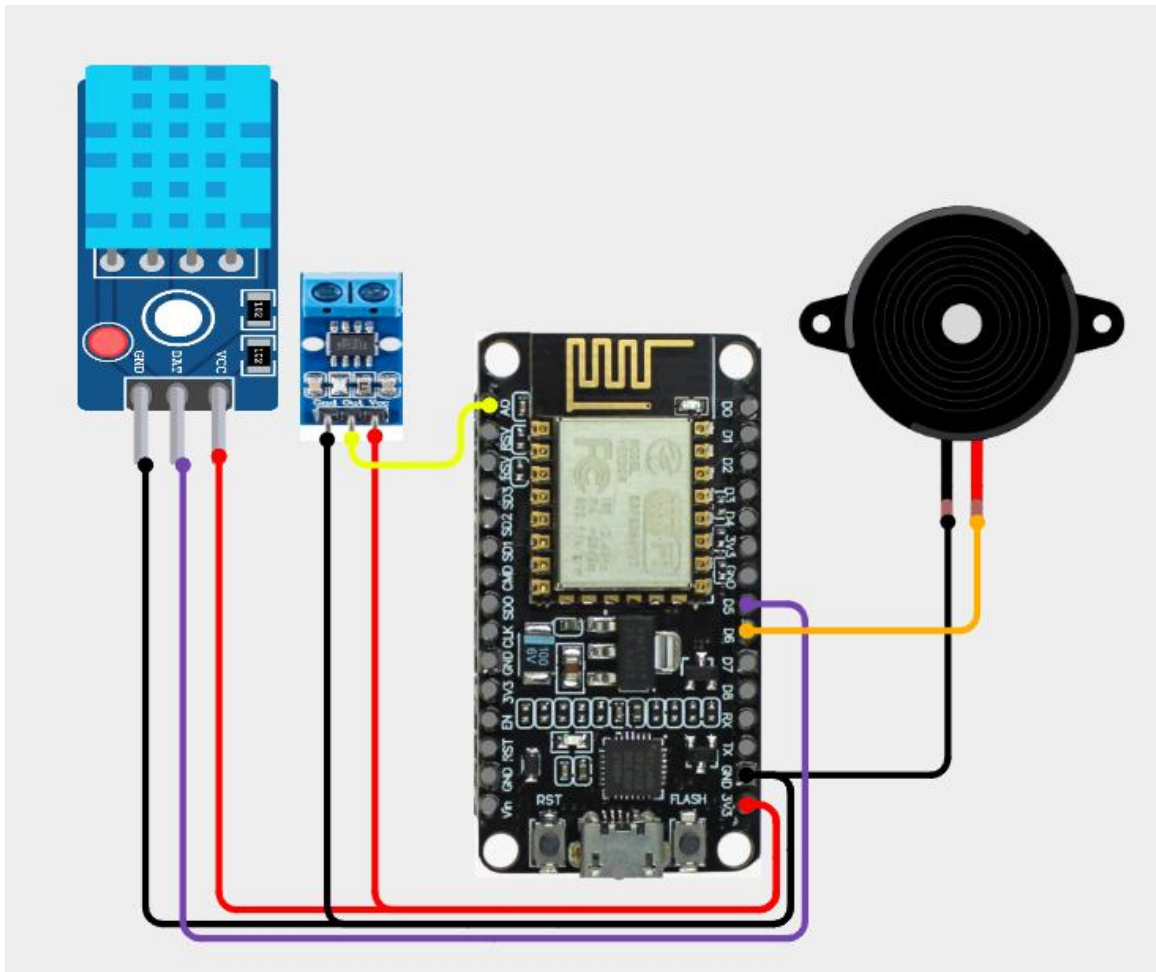
- Sensor data acquisition
- Analog signal processing
- Temperature processing
- Humidity processing
- Voltage evaluation
- Current evaluation
- Load calculation
- Threshold comparison
- Fault detection
- Alarm control
- OLED updating
- System status generation

The proposed Industrial Diesel Generator Monitoring and Maintenance System consists of a sensing layer, processing layer, display layer, and alert layer. The sensing layer consists of an ACS712 current sensor, voltage sensor module, and DHT11 temperature and humidity sensor. These sensors collect real-time information related to the electrical and environmental operating conditions of the generator. The sensor outputs are supplied to the ESP8266 NodeMCU, which acts as the central processing and control unit. The ESP8266 processes the acquired data and compares the measured parameters with predefined threshold values. Based on the comparison, the controller determines whether the generator is operating under normal, warning, or critical conditions. The OLED display provides real-time information about voltage, current, temperature, humidity, and system status. A buzzer is used to provide an audible warning when an abnormal condition such as excessive current, abnormal voltage, or high temperature is detected. The resulting system provides continuous local monitoring and assists operators in identifying abnormal operating conditions at an early stage, thereby supporting preventive maintenance and improving generator operational reliability.

Circuit Table:

Component	Pin	ESP8266 Pin	Function
DHT11	DATA	D4	Temperature
ACS712	OUT	A0	Current measurement
Voltage Sensor	OUT	A0*	Voltage measurement
OLED	SDA	D2	I ² C data
OLED	SCL	D1	I ² C clock
DHT11	VCC	3.3V	Power
DHT11	GND	GND	Ground

Circuit Design:



Algorithm:

- Start.
- Initialize ESP8266.
- Initialize OLED.
- Initialize DHT11.
- Initialize buzzer.
- Read temperature.
- Read humidity.
- Read current.
- Read voltage.

- Calculate electrical load.
- Compare temperature with threshold.
- Compare current with rated current.
- Compare voltage with acceptable voltage range.
- Determine generator status.
- Display values on OLED.
- If values are normal, keep buzzer OFF.
- If warning condition is detected, activate intermittent buzzer.
- If critical condition is detected, activate continuous buzzer.
- Repeat the process continuously.

Coding:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <DHT.h>
#define WIFI_SSID "K-APIL"
#define WIFI_PASS "kapil.2333"
```

```
#define IO_USERNAME "saravanakarthiks"
#define IO_KEY      "aio_idPg82y91fmNVXL0iJA71zKwH97q"
```

```
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
```

```
AdafruitIO_Feed *currentFeed =
  io.feed("generator-current");
```

```
AdafruitIO_Feed *temperatureFeed =
  io.feed("generator-temperature");
```

```
AdafruitIO_Feed *humidityFeed =
  io.feed("generator-humidity");
```

```
AdafruitIO_Feed *statusFeed =
  io.feed("generator-status");
```

```
AdafruitIO_Feed *buzzerFeed =
```

```
io.feed("generator-buzzer");

#define ACS_PIN    A0
#define DHT_PIN    14    // D5 = GPIO14
#define BUZZER_PIN 12    // D6 = GPIO12

#define DHTTYPE DHT11

DHT dht(DHT_PIN, DHTTYPE);

const float MAX_CURRENT = 10.0; // Amperes
const float MAX_TEMP    = 50.0; // Celsius
const float MAX_HUMIDITY = 80.0; // Percentage

// ACS712-5A = 0.185
// ACS712-20A = 0.100
// ACS712-30A = 0.066

const float ACS_SENSITIVITY = 0.100;

// Zero-current voltage
const float ACS_ZERO_VOLTAGE = 2.50;

// ESP8266 ADC voltage
const float ADC_VOLTAGE = 3.3;

float current = 0;
float temperature = 0;
float humidity = 0;

String generatorStatus;

unsigned long previousMillis = 0;

const unsigned long interval = 5000;

void setup()
{
    Serial.begin(115200);
```



```

delay(1000);

Serial.println();
Serial.println("=====");
Serial.println(" INDUSTRIAL DIESEL GENERATOR MONITORING");
Serial.println("=====");

pinMode(BUZZER_PIN, OUTPUT);

digitalWrite(BUZZER_PIN, LOW);

dht.begin();

Serial.println("DHT11 Started");
Serial.println("ACS712 Started"); Serial.println();
Serial.println("Connecting to Adafruit IO...");

io.connect();

while (io.status() < AIO_CONNECTED)
{
    Serial.print(".");
    delay(500);
}

Serial.println();
Serial.println("Adafruit IO Connected!");

Serial.println("-----");

Serial.print("Current Limit   : ");
Serial.print(MAX_CURRENT);
Serial.println(" A");

Serial.print("Temperature Limit : ");
Serial.print(MAX_TEMP);
Serial.println(" C");

Serial.print("Humidity Limit   : ");
Serial.print(MAX_HUMIDITY);
Serial.println(" %");

Serial.println("-----");
}

```

```
// READ ACS712 CURRENT
```

```
float readCurrent()
{
    const int samples = 100;

    float voltageSum = 0;

    for (int i = 0; i < samples; i++)
    {
        int adcValue = analogRead(ACS_PIN);

        float sensorVoltage =
            (adcValue / 1023.0) * ADC_VOLTAGE;

        voltageSum += sensorVoltage;

        delay(2);
    }

    float averageVoltage =
        voltageSum / samples;

    float currentValue =
        (averageVoltage - ACS_ZERO_VOLTAGE)
        / ACS_SENSITIVITY;

    // Convert negative to positive
    if (currentValue < 0)
    {
        currentValue = -currentValue;
    }

    // Remove small noise
    if (currentValue < 0.15)
    {
        currentValue = 0;
    }

    return currentValue;
}
```

```
void readSensors()
```

```
{
  // DHT11
  temperature = dht.readTemperature();
  humidity = dht.readHumidity();

  // ACS712
  current = readCurrent();
}
```

```
void checkStatus()
{
  bool currentHigh =
    current > MAX_CURRENT;

  bool temperatureHigh =
    temperature > MAX_TEMP;

  bool humidityHigh =
    humidity > MAX_HUMIDITY;
```

```
  if (currentHigh &&
      temperatureHigh &&
      humidityHigh)
  {
    generatorStatus = "CRITICAL";

    digitalWrite(BUZZER_PIN, HIGH);
  }
```

```
  else if (currentHigh &&
           temperatureHigh)
  {
    generatorStatus = "OVERLOAD + HIGH TEMP";

    digitalWrite(BUZZER_PIN, HIGH);
  }
```

```
  else if (currentHigh)
  {
    generatorStatus = "OVERLOAD";
```

```
    digitalWrite(BUZZER_PIN, HIGH);  
}
```

```
else if (temperatureHigh)  
{  
    generatorStatus = "HIGH TEMPERATURE";  
  
    digitalWrite(BUZZER_PIN, HIGH);  
}
```

```
else if (humidityHigh)  
{  
    generatorStatus = "HIGH HUMIDITY";  
  
    digitalWrite(BUZZER_PIN, HIGH);  
}
```

```
else  
{  
    generatorStatus = "NORMAL";  
  
    digitalWrite(BUZZER_PIN, LOW);  
}  
}
```

```
void sendToAdafruit()  
{  
    // Current  
    currentFeed->save(current);  
  
    // Temperature  
    temperatureFeed->save(temperature);  
  
    // Humidity  
    humidityFeed->save(humidity);  
  
    // Status  
    statusFeed->save(generatorStatus.c_str());  
  
    // Buzzer  
    if (generatorStatus == "NORMAL")
```

```

    {
        buzzerFeed->save("OFF");
    }
    else
    {
        buzzerFeed->save("ON");
    }

    Serial.println();
    Serial.println("Data sent to Adafruit IO");
}

void printSerialData()
{
    Serial.println();
    Serial.println("=====");
    Serial.println("    DIESEL GENERATOR STATUS");
    Serial.println("=====");

    Serial.print("Current      : ");
    Serial.print(current, 2);
    Serial.println(" A");

    Serial.print("Temperature   : ");
    Serial.print(temperature, 1);
    Serial.println(" C");

    Serial.print("Humidity      : ");
    Serial.print(humidity, 1);
    Serial.println(" %");

    Serial.println("-----");

    Serial.print("Current Limit  : ");
    Serial.print(MAX_CURRENT, 2);
    Serial.println(" A");

    Serial.print("Temperature Limit: ");
    Serial.print(MAX_TEMP, 1);
    Serial.println(" C");

    Serial.print("Humidity Limit  : ");
    Serial.print(MAX_HUMIDITY, 1);
    Serial.println(" %");

    Serial.println("-----");

```

```
Serial.print("Generator Status : ");
Serial.println(generatorStatus);

Serial.print("Buzzer      : ");

if (generatorStatus == "NORMAL")
{
    Serial.println("OFF");
}
else
{
    Serial.println("ON");
}

Serial.println("=====");
}

void loop()
{
    // Keep Adafruit IO connection alive
    io.run();

    unsigned long currentMillis = millis();

    if (currentMillis - previousMillis >= interval)
    {
        previousMillis = currentMillis;

        readSensors();

        if (isnan(temperature) ||
            isnan(humidity))
        {
            Serial.println("ERROR: DHT11 reading failed!");

            digitalWrite(BUZZER_PIN, HIGH);

            delay(300);

            digitalWrite(BUZZER_PIN, LOW);

            return;
        }
    }
}
```

```
}
```

```
checkStatus();
```

```
printSerialData();
```

```
sendToAdafruit();
```

```
}
```

```
}
```

System Working:

Step 1 — Power ON

When the system is switched on, the ESP8266 initializes the sensors and OLED.

The display shows:

INDUSTRIAL

GENERATOR

MONITORING SYSTEM

Step 2 — Temperature and Humidity Measurement

The DHT11 measures environmental temperature and humidity.

Example:

Temperature = 32°C

Humidity = 60%

The information is sent to the ESP8266.

Step 3 — Current Measurement

The ACS712 measures the current flowing through the monitored circuit.

Example:

Current = 12 A

If the generator's rated current is 20 A:

Load % = $(12 / 20) \times 100$

Load = 60%

Therefore:

STATUS = NORMAL

Step 4 — Voltage Measurement

The voltage sensor provides a scaled representation of the measured voltage.

Example:

Generator Voltage = 230 V

If the acceptable range is:

210 V – 250 V

then:

230 V → NORMAL

Step 5 — Fault Detection

The ESP8266 compares the readings against thresholds.

Example 1

Voltage = 230 V

Current = 12 A

Temperature = 32°C

Humidity = 60%

Result:

STATUS: NORMAL

BUZZER: OFF

Example 2

Current = 22 A

Rated Current = 20 A

Result:

STATUS: OVERLOAD

BUZZER: ON

Example 3

Temperature = 45°C

Result:

STATUS: TEMP WARNING

BUZZER: INTERMITTENT

```
=====
DIESEL GENERATOR STATUS
=====
```

```
Current      : 24.94 A
Temperature  : 28.3 C
Humidity     : 72.1 %
-----
```

```
Current Limit : 10.00 A
Temperature Limit: 50.0 C
Humidity Limit : 80.0 %
-----
```

```
Generator Status : OVERLOAD
Buzzer           : ON
=====
```

```
Data sent to Adafruit IO
```

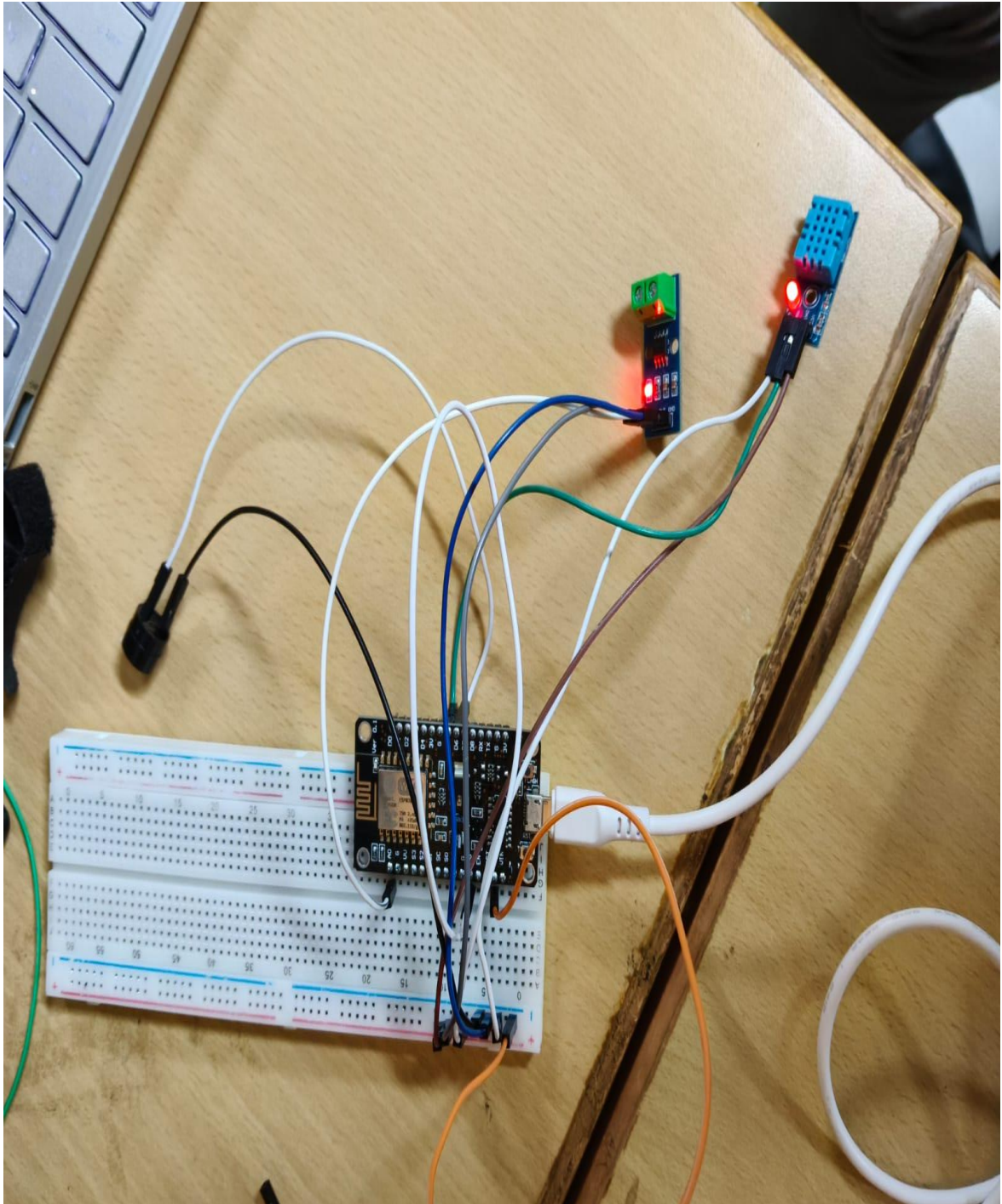
```
=====
DIESEL GENERATOR STATUS
=====
```

```
Current      : 24.93 A
Temperature  : 28.3 C
Humidity     : 72.1 %
-----
```

```
Current Limit : 10.00 A
Temperature Limit: 50.0 C
Humidity Limit : 80.0 %
-----
```

```
Generator Status : OVERLOAD
Buzzer           : ON
=====
```

Prototype Demonstration:



Results & Performance Analysis:

The developed system should successfully:

- Measure generator current.
- Measure generator voltage.
- Measure ambient temperature.
- Measure humidity.
- Display measurements on OLED.
- Detect excessive current.
- Detect abnormal voltage.
- Detect high temperature.
- Generate an audible warning.
- Provide real-time generator status.

The materials you listed are sufficient for a **prototype**, but there are some important limitations for an actual industrial generator.

1. DHT11

The DHT11 measures ambient temperature and humidity. It is **not a substitute for an engine coolant-temperature or oil-temperature sensor**.

2. Single Analog Input

The ESP8266 NodeMCU has one analog input, so the ACS712 and voltage sensor cannot independently occupy the same A0 input simultaneously without additional hardware.

3. Industrial Voltage

A normal voltage-sensor module should not simply be connected to an industrial generator's high-voltage output.

4. ACS712

The ACS712 version must match the expected current range, and the sensor must be installed according to its electrical rating.

5. Maintenance Monitoring

With the listed components, the system can provide **condition-based warning information**, but it does not have a dedicated real-time clock or nonvolatile maintenance database. For a more complete maintenance system, an RTC and storage/cloud system could be added later.